ENSEEIHT / 1SN

Technologie Objet / Projet Long

Rapport n°-3

MANAG'EAT

Equipe EF-2

Alvarez Tommy, Boutel William
Lucas Ewen, Meunier Lery, Prum
Matis, Tamboura Samir, Naime
Achraf, Benhmidane Driss

SOMMAIRE

I. INTRODUCTION GÉNÉRALE……..……3

A. Rappel du sujet…..……3

II. PRINCIPALES FONCTIONNALITÉS……....4

A. Chronologie du développement et cycle des Sprints ……4

B. Tableau de suivi et d'état d'avancement des fonctionnalités …..5

C. Bilan des réalisations et arbitrages.....6

III. DÉCOUPAGE DE L'APPLICATION……7

A. Le paquetage "modele"……..……7

B. Le paquetage "vue".....……..7

C. Le paquetage "controleur".....8

D. Le paquetage "Dao".....8

IV. CONCEPTION STATIQUE..……..……..9

A. Modélisation UML de la structure logicielle ...……9

B. Domaine métier et le module RH (modele) ……..……9

C. Implémentation du patron de conception State……10

D. Synthèse technique et cartographie complète des classes…….11

V. CONCEPTION DYNAMIQUE ……..……..……..13

A. Scénario d'authentification et contrôle d'accès (RBAC)……13

B. Prise de commande en salle et envoi en cuisine……...14

C. Cycle de persistance et gestion du stockage JSON..……15

VI. CHOIX DE CONCEPTION, DIFFICULTÉS ET SOLUTIONS …...…..16

A. Principaux choix de conception et de réalisation ……16

B. Problèmes techniques rencontrés et solutions apportées …..…16

VII. ORGANISATION DE L'ÉQUIPE ET MÉTHODOLOGIE AGILE ..…...18

A. Introduction et structuration du Backlog ……18

B. Communication, exécution et transparence opérationnelle…….18

C. Gestion de Git et protocole de prévention des conflits ……19

D. Documentation et livrables communs ……19

VIII. CONCLUSION……..…...20

I. INTRODUCTION GÉNÉRALE

A. Rappel du sujet

Le projet Manag'Eat est né d'un constat partagé sur les limites des outils informatiques actuellement déployés dans le secteur de la restauration. La majorité des solutions existantes souffrent d'une forte segmentation : les équipes de direction et le personnel en salle doivent constamment naviguer entre des logiciels distincts pour la facturation, la planification des ressources humaines et la gestion physique des tables. Cette dispersion des flux de données engendre des risques élevés d'erreurs de saisie, des pertes de temps chroniques et un manque de visibilité globale sur l'activité, particulièrement dommageable lors des périodes de fort trafic.

Pour répondre à ces enjeux, l'application Manag'Eat centralise l'intégralité du cycle opérationnel d'un établissement au sein d'une architecture unique, fluide et modulaire, articulée autour de trois axes fondamentaux :

- **La gestion de salle et l'ingénierie d'espace :** À travers une cartographie interactive et dynamique (couvrant la terrasse, le rez-de-chaussée et les étages), le système offre un suivi en temps réel du cycle de vie des tables (libres, réservées, occupées) et optimise la rotation de la clientèle.
- **Le pilotage des ressources humaines :** Un module de planification basé sur une frise chronologique interactive permet d'ajuster en continu les effectifs (serveurs et cuisiniers) face aux pics d'affluence prévisibles, tout en intégrant un contrôle strict des contraintes légales de travail.
- **La prise de commande et la gestion des flux financiers :** Un panneau de saisie intuitif fluidifie la transmission des commandes en cuisine, intègre des mécanismes anti-gaspillage automatisés (les "Menus Flash") et simplifie l'encaissement grâce à un outil de division de note individualisé.

L'objectif de ce rapport est de synthétiser les choix architecturaux, les spécifications techniques et les méthodologies agiles qui ont jalonné le développement de l'application.

Les détails fonctionnels complets (scénarios d'utilisation, captures d'écran, guide pas-à-pas) sont présentés dans le manuel Utilisateur joint à ce rapport.

II. PRINCIPALES FONCTIONNALITÉS DE L'APPLICATION

A. Chronologie du développement et cycle des Sprints

L'application Manag'Eat s'articule autour de six "épics" correspondant aux grands modules fonctionnels du logiciel.

Le premier épic, **pilotage de la salle et prise de commande**, a été entièrement réalisé lors des itérations 1 et 3. Il couvre l'affichage d'un plan de salle interactif gérant les états des tables, la prise de commande avec recherche filtrée et notes spéciales, la gestion des zones de service par serveur, le module de paiement avec division d'addition, ainsi que l'envoi des commandes en cuisine, finalisé en itération 3.

Le deuxième épic, **gestion RH et planning**, est également terminé (itérations 1 et 2). Il comprend un calendrier interactif avec drag & drop des créneaux, la gestion CRUD des employés, des alertes visuelles de dépassement d'heures, la conversion automatique du planning en données de paie, le suivi des heures supplémentaires et du taux d'absentéisme, ainsi qu'un algorithme de rotation suggérée des serveurs.

Le troisième épic, **persistance des données et authentification**, a été développé en itérations 2 et 3. Il repose sur la sérialisation JSON via Gson, un système de connexion par rôles (GERANT_CHAINE, RH, SERVEUR, CUISINIER) reconfigurant dynamiquement l'interface, un assistant de configuration au premier lancement, la persistance des commandes et le cloisonnement strict des données par établissement.

Le quatrième épic, **gestion multi-restaurants et carte de France**, est terminé en itérations 2 et 3. Il permet d'ajouter ou supprimer des restaurants sur une carte interactive, de saisir un chiffre d'affaires prévisionnel par établissement, et assure une isolation complète des données entre restaurants.

Le cinquième épic, **module cuisine**, a été intégralement réalisé en itération 3. Il offre aux cuisiniers une file d'attente des commandes, une gestion des stocks avec jauge de disponibilité et bouton « Rupture », ainsi qu'une synchronisation en temps réel avec la salle.

Le sixième épic, **statistiques et aide à la décision**, n'est que partiellement réalisé. Les KPI RH (turnover, absentéisme, productivité) et le tableau de suivi individuel des serveurs ont été développés en itération 2, et les statistiques globales de la chaîne ont été amorcées en itération 3. En revanche, le simulateur de rentabilité, le moteur de prédiction des flux et le système de menus flash à tarification dynamique n'ont pas été réalisés, les efforts ayant été concentrés sur la consolidation de l'architecture et du cœur opérationnel de l'application.

B. Tableau de suivi et d'état d'avancement des fonctionnalités

Épic	User story	État	Itération
Épic 1Salle &commande	Plan de salle interactif (états des tables)	**Terminé**	It. 1
	Sélection des tables par clic	**Terminé**	It. 1
	Prise de commande avec recherche filtrée et notes spéciales	**Terminé**	It. 1
	Gestion des zones de service par serveur	**Terminé**	It. 1
	Module de paiement et division d'addition	**Terminé**	It. 1
	Envoi de commande en cuisine	**Terminé**	It. 3
Épic 2 RH &planning	Calendrier interactif avec drag & drop des créneaux	**Terminé**	It. 1
	CRUD employés	**Terminé**	It. 1
	Alertes de dépassement d'heures et de pauses	**Terminé**	It. 1
	Conversion planning → données de paie	**Terminé**	It. 2
	Suivi heures supplémentaires et absentéisme	**Terminé**	It. 2
	Algorithme de rotation des serveurs	**Terminé**	It. 2
Épic 3 Persistance& auth	Sauvegarde/chargement JSON (Gson)	**Terminé**	It. 2
	Connexion par rôles (RBAC)	**Terminé**	It. 2
	Interface dynamique selon le profil connecté	**Terminé**	It. 2
	Assistant de configuration au premier lancement	**Terminé**	It. 2
	Persistance des commandes	**Terminé**	It. 3
	Cloisonnement des données par restaurant	**Terminé**	It. 3
Épic 4 Multi-restaurants	Carte de France interactive (ajout/suppression de restaurants)	**Terminé**	It. 2
	Chiffre d'affaires prévisionnel par restaurant	**Terminé**	It. 2
	Sélection du restaurant actif	**Terminé**	It. 2
	Isolation complète des données par établissement	**Terminé**	It. 3
Épic 5 Module cuisine	File d'attente des commandes	**Terminé**	It. 3
	Gestion des stocks avec jauge et bouton « Rupture »	**Terminé**	It. 3
	Synchronisation en temps réel salle ↔ cuisine	**Terminé**	It. 3
Épic 6 Statistiques	KPI RH (turnover, absentéisme, productivité)	**Terminé**	It. 2
	Tableau de suivi individuel des serveurs	**Terminé**	It. 2
	Statistiques globales de la chaîne	**Commencé**	It. 3
	Simulateur de rentabilité	**Non réalisé**	—
	Moteur de prédiction des flux	**Non réalisé**	—
	Menus Flash et tarification dynamique	**Non réalisé**	—

C. Bilan des réalisations et arbitrages

L'analyse de ce cycle de développement met en évidence une progression logique : les efforts initiaux se sont portés sur le cœur opérationnel de terrain (salle et ressources humaines) avant d'étendre le périmètre aux fonctionnalités de supervision (cuisine, statistiques) et de persistance. Les fonctionnalités d'aide à la décision non réalisées ont fait l'objet d'un arbitrage conscient afin de sécuriser la stabilité et la robustesse (principalement la correction des bugs de l’application) de l'architecture centrale face aux contraintes de temps.

III. DÉCOUPAGE DE L'APPLICATION

Pour garantir la maintenabilité, l'évolutivité et la robustesse de l'application Manag'Eat, le code source a été structuré selon le patron d'architecture MVC (Modèle-Vue-Contrôleur), complété par une couche de persistance DAO (Data Access Object). Ce découpage permet une séparation stricte des préoccupations, isolant la logique métier des détails de l'interface graphique ou des mécanismes de stockage.

L'application est ainsi segmentée en quatre paquetages (*packages*) principaux, interconnectés mais indépendants.

A. Le paquetage "modele"

Ce paquetage contient l'ensemble des concepts physiques et logiques propres au domaine de la restauration. Les classes de ce module sont purement de nature logicielle et métier ; elles ne possèdent aucune dépendance vis-à-vis des bibliothèques graphiques (Java Swing), ce qui assure leur portabilité. Ce package sert notamment à représenter les données de l'application et encapsuler les règles métier (par exemple, interdire la réservation d'une table déjà occupée ou valider qu'un employé ne dépasse pas son quota d'heures réglementaires).

Classes principales : `Restaurant`, `Table`, `Reservation`, `Employe`, `Planning`, `Creneau`, `Commande`, `Plat`.

B. Le paquetage "vue" (Interface Homme-Machine)

La vue regroupe tous les composants graphiques qui matérialisent l'écran de l'utilisateur. Développée intégralement à l'aide de l'API Java Swing, elle s'appuie sur une hiérarchie de conteneurs (`JFrame` pour la fenêtre principale et différents `JPanel` pour les modules). Ce package permet de présenter les données du modèle à l'utilisateur sous une forme graphique (les onglets, la frise chronologique du planning, le plan de salle interactif) et intercepter les clics ou actions physiques de l'utilisateur. Les vues sont passives : elles ne modifient jamais les données directement.

Classes principales : `FenetrePrincipale`, `PanelDashboard`, `PanelGestionnaire (RH)`, `PanelStatistiques`, `PanelParametres`.

C. Le paquetage “controleur” (Le Cerveau de l'Application)

Le contrôleur sert de pont entre la vue et le modèle. Il écoute les actions de l'utilisateur capturées par la vue, exécute la logique métier correspondante en manipulant le modèle, puis demande à la vue de se rafraîchir. Le package contribue à orchestrer les flux d'informations et réagir aux événements de l'interface. C'est ici que sont implémentés les mécanismes de sécurité, comme le contrôle d'accès basé sur les rôles pour n'afficher que les onglets autorisés au profil connecté.

Classes principales : `ControleurGeneral`, `ActionPlanificationHandler`, `CommandeController`, `AuthentificationManager`.

D. Le paquetage “dao” (La Persistance des Données)

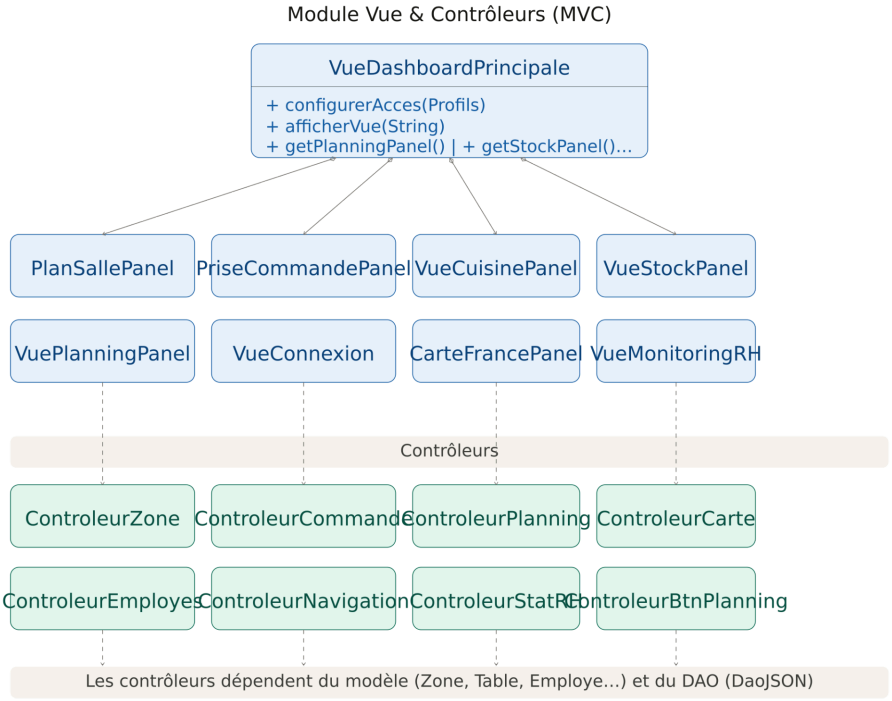
La couche DAO (*Data Access Object*) isole la gestion des données de l'application du reste du code. Plutôt que de laisser les contrôleurs manipuler directement des fichiers, le reste de l'application passe par des interfaces DAO génériques. Le package gère la sauvegarde et le chargement des données. C'est ce paquetage qui charge le fichier central `carte_restaurants_complets.json` et qui assure le cloisonnement strict des données pour chaque établissement.

Classes principales : `RestaurantDAO`, `JsonManager`.

IV. CONCEPTION STATIQUE

A. Modélisation UML de la structure logicielle

Afin d'avoir une vision macroscopique du logiciel, nous avons d'abord schématisé l'interconnexion globale de nos packages. Cela permet de visualiser immédiatement comment l'interface utilisateur, la logique de contrôle et le stockage de données collaborent sans s'entremêler.



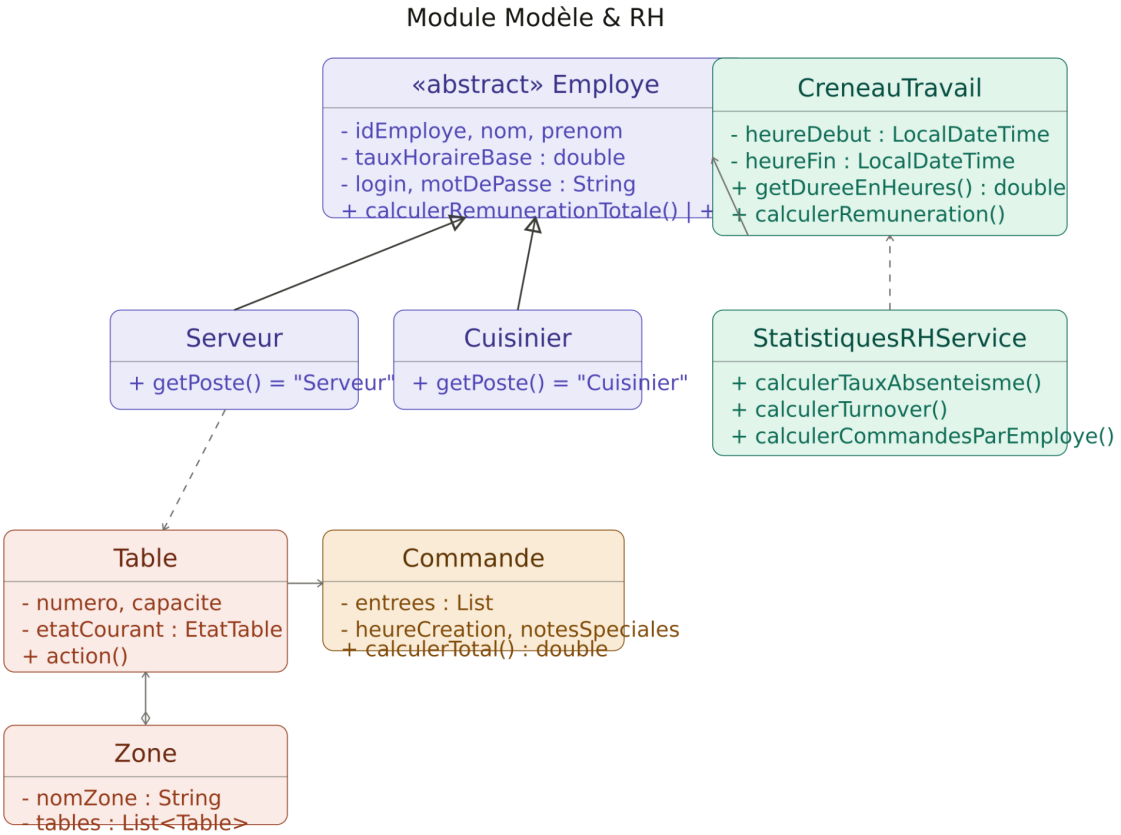
B. Domaine métier et le module RH (modele)

En plongeant dans le cœur de notre application, nous avons structuré nos classes pour modéliser précisément les ressources et l'activité d'une chaîne de restauration :

- **Gestion du personnel :** Nous avons créé une classe abstraite *Employe* qui centralise les propriétés communes à tout le personnel (*idEmploye*, *nom*, *prenom*, *tauxHoraireBase*, etc.). Nous nous appuyons ensuite sur le polymorphisme avec les sous-classes *Serveur* et *Cuisinier* pour spécialiser les rôles via la méthode *getPoste*().

- **Planification et Suivi :** Les objets *CreneauTravail* lient un employé à une plage horaire (*LocalDateTime*). Pour ne pas surcharger nos classes de données pures, nous avons choisi de déléguer entièrement l'extraction des indicateurs de performance à une classe de service dédiée, *StatistiquesRHService* (*calculerTauxAbsenteisme*(), *calculerTurnover*()).

- **Structure physique et ventes :** Notre classe principale Restaurant agrège des entités *Zone* (Terrasse, RDC), elles-mêmes composées de structures Table. Les flux de vente transitent par la classe *Commande*, qui est composée d'une liste d'objets *EntreeCommande* afin de gérer l'association avec un *Plat* ou une *Formule*.



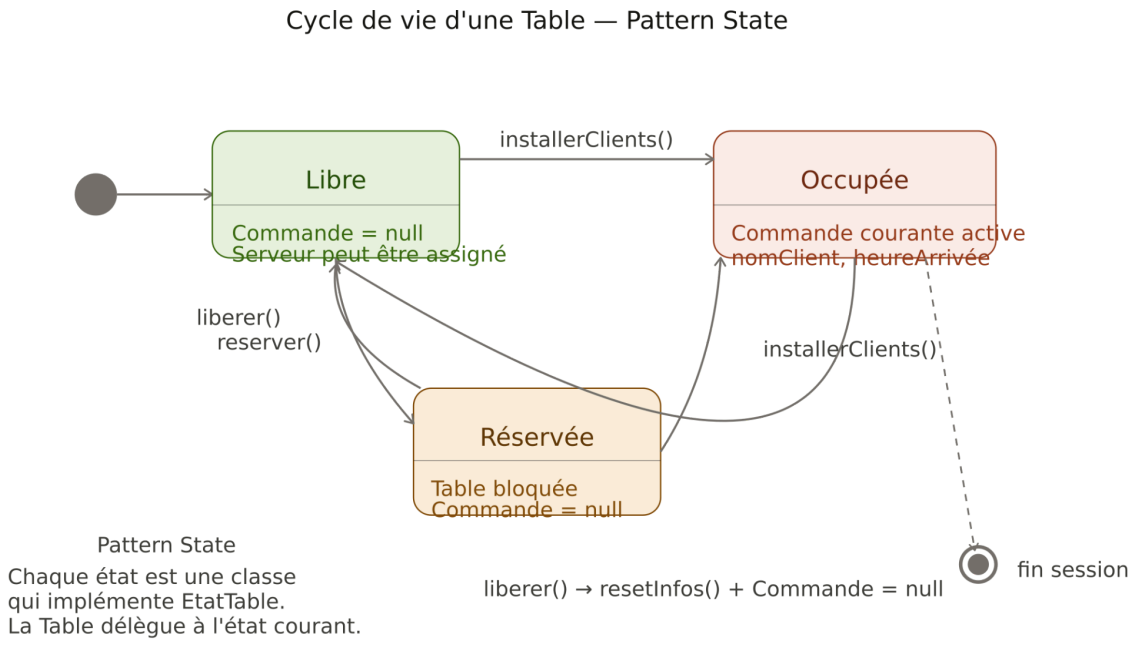
C. Implémentation du patron de conception State

Afin d'éviter d'écrire des structures conditionnelles complexes (*if/else* ou *switch* imbriqués) lors du changement de statut des tables en salle, nous avons fait le choix d'implémenter le patron de conception State.

La gestion de la dynamicité de nos tables est ainsi déléguée à une interface EtatTable. Cette interface est réalisée par trois classes concrètes qui représentent les différents états mutables de notre système : *EtatLibre*, *EtatReservee*, et *EtatOccupee*.

- **EtatLibre :** L'instance de *Commande* est initialisée à *null* et un serveur peut être librement assigné à la zone.
- **EtatReservee :** La table est bloquée logiquement, ce qui restreint l'accès à la prise de commande immédiate.
- **EtatOccupee :** La table pointe vers une instance de Commande courante active, encapsulant le *nomClient* et *l'heureArrivee*.

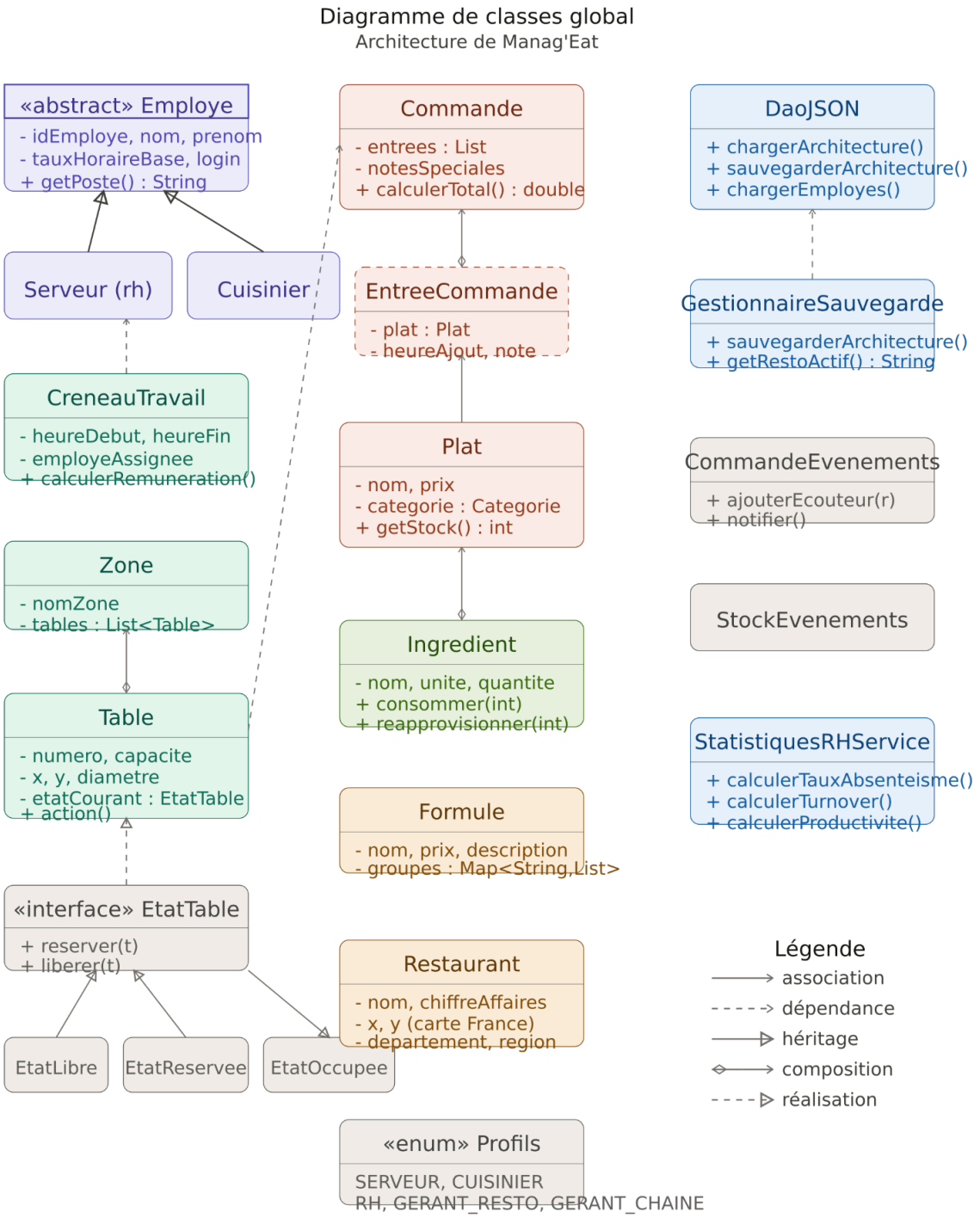
Les transitions d'état sont déclenchées par nos méthodes d'action (*reserver*(), *installerClients*(), *liberer*()). Par exemple, lors de l'appel de *liberer*(), la table réinitialise ses attributs via *resetInfos*(), repasse la commande active à *null* et bascule à nouveau automatiquement vers l'état *EtatLibre*.



D. Synthèse technique et cartographie complète des classes

Pour conclure sur la dimension structurelle de Manag'Eat, nous avons regroupé l'ensemble des interactions du logiciel dans une cartographie globale. Ce rendu final met en avant les attributs majeurs et les relations d'association et de composition indispensables à la compréhension du projet, en omettant

volontairement les méthodes secondaires pour ne pas surcharger la lecture.



V. CONCEPTION DYNAMIQUE

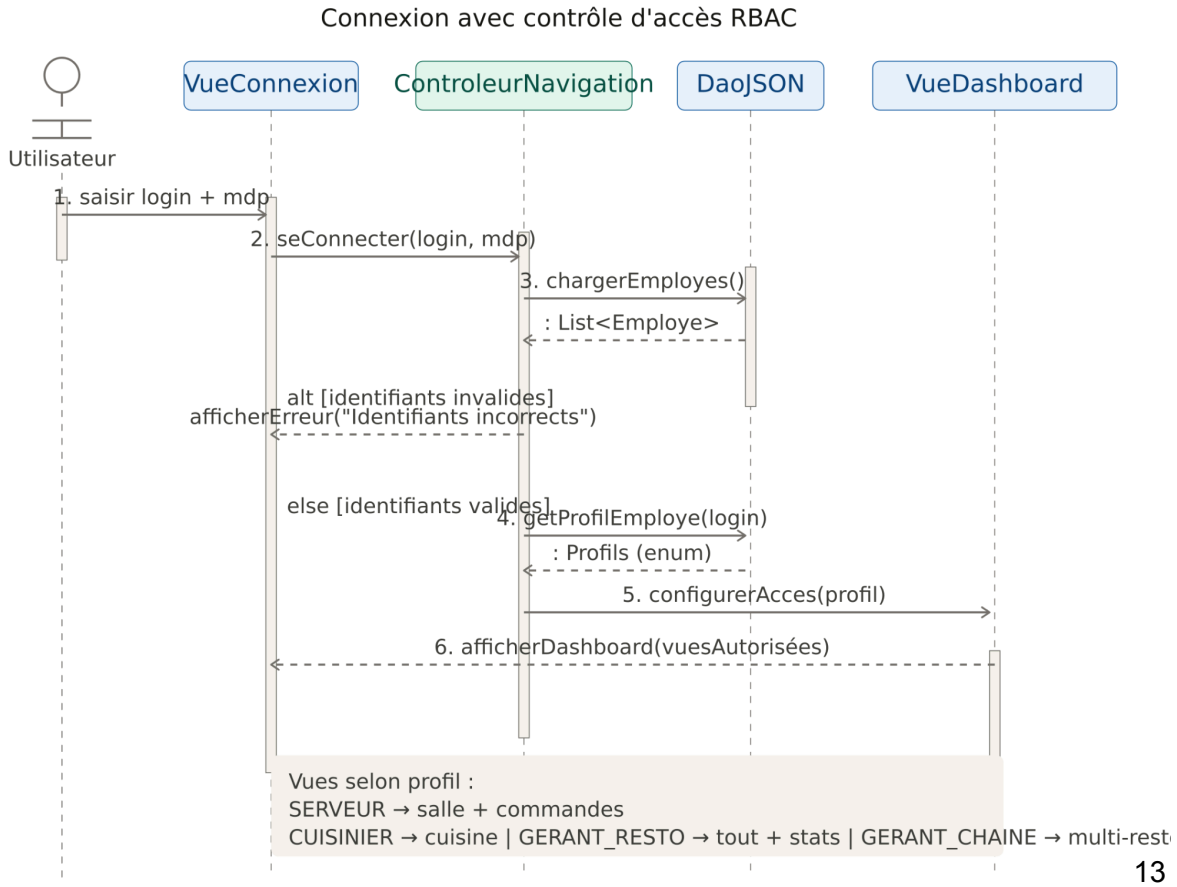
A. *Scénario d'authentification et contrôle d'accès (RBAC)*

Le premier mécanisme critique de notre application est le filtrage des utilisateurs dès le démarrage. Nous avons mis en place un contrôle d'accès basé sur les rôles (RBAC) pour configurer l'interface de manière dynamique.

Le diagramme de séquence montre que lorsque l'utilisateur saisit ses identifiants sur l'IHM *VueConnexion*, celle-ci délègue l'action au *ControleurNavigation* via l'appel de *seConnecter*(login, mdp). Le contrôleur interroge alors la couche de persistance *DaoJSON* pour charger la liste des employés du restaurant. Deux alternatives (alt) se présentent :

1. **Identifiants invalides :** Le contrôleur retourne un message d'erreur et l'IHM affiche immédiatement une alerte visuelle.

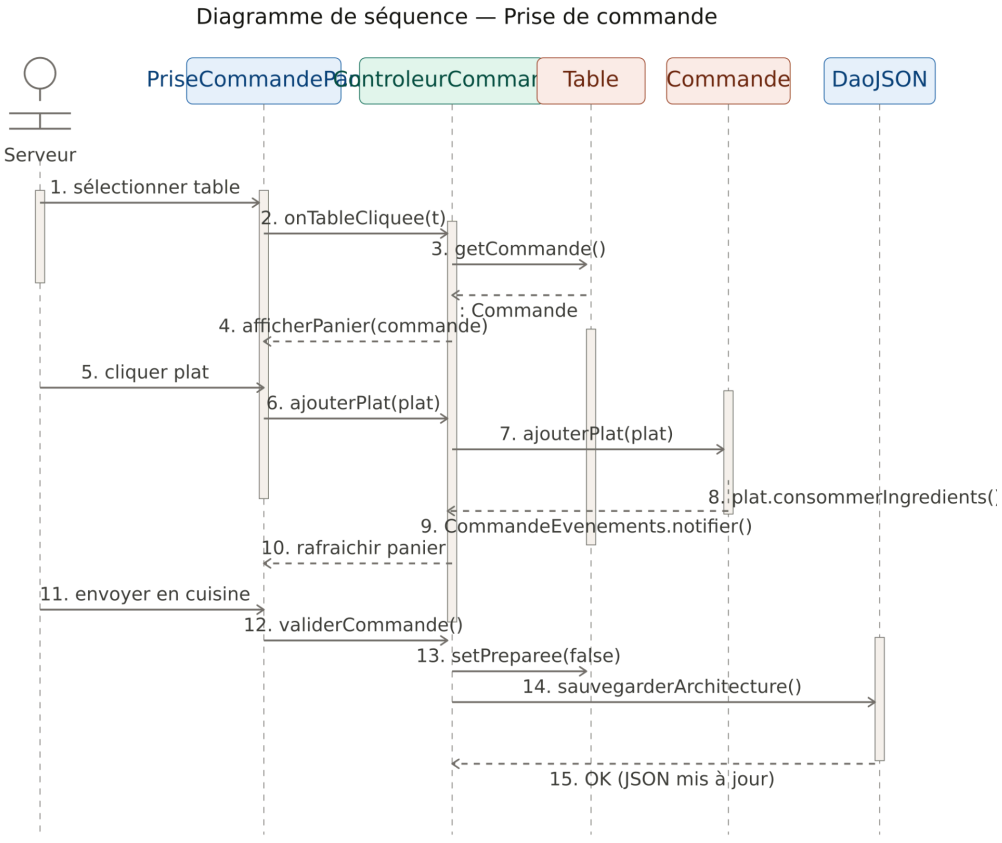
2. **Identifiants valides :** Le contrôleur récupère le profil typé de l'employé (via l'énumération Profils), puis appelle la méthode *configurerAcces*(profil) sur notre *VueDashboard*. C'est ce message clé qui active uniquement les onglets autorisés (par exemple, restreindre le cuisinier à l'écran de cuisine et le serveur aux fonctionnalités de salle).



B. Scénario opérationnel : Prise de commande en salle et envoi en cuisine

Ce diagramme détaille l'enchaînement des interactions en temps réel lorsqu'un serveur s'occupe d'une table en salle. Le flux d'exécution met en avant notre découpage strict et la mise à jour asynchrone des composants :

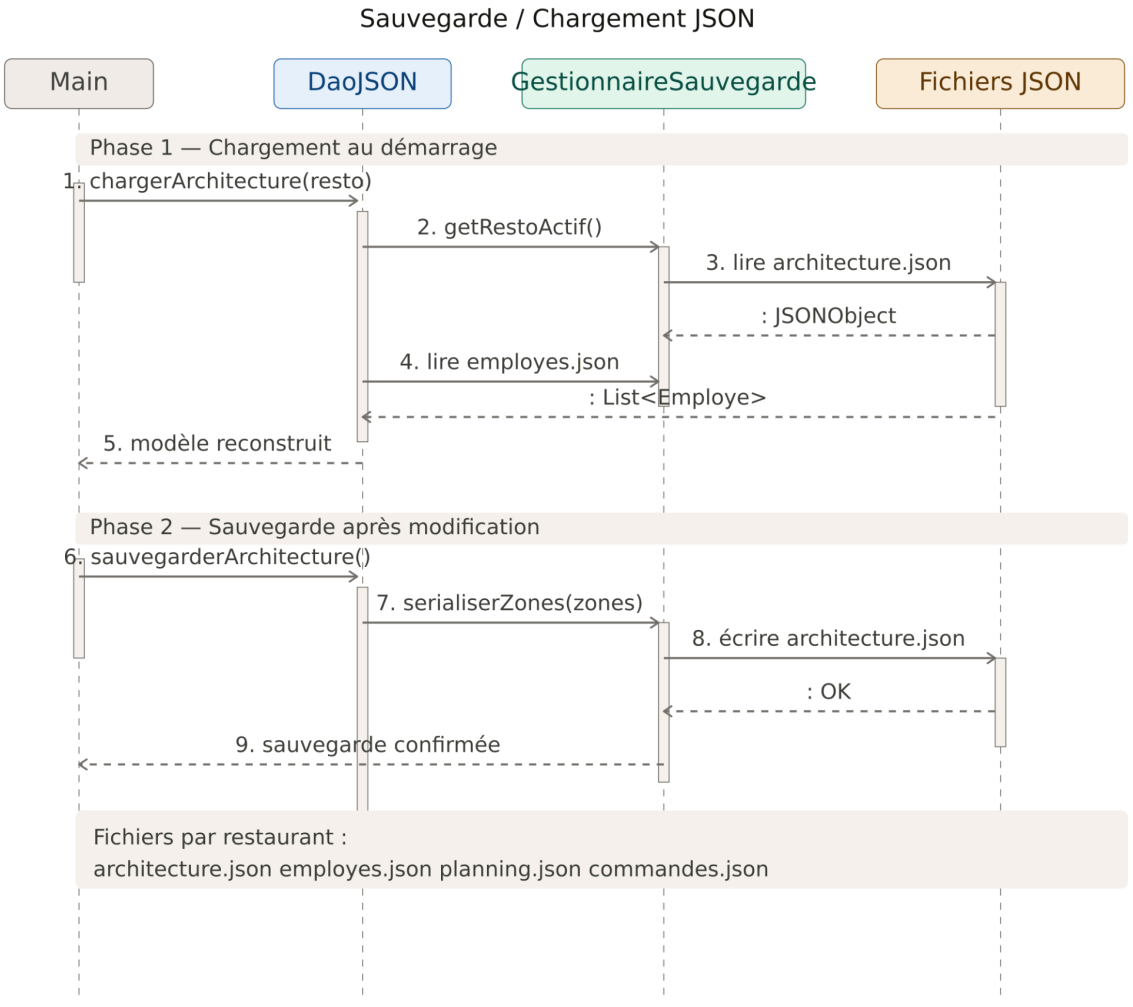
1. **Sélection et affichage :** Lorsque le serveur clique sur une table de l'interface *PriseCommandePanel*, l'événement est intercepté par le *ControleurCommande* qui demande l'objet *Commande* associé à l'entité *Table*. L'IHM récupère ces informations pour afficher graphiquement le panier du client.
2. **Ajout d'articles et gestion des stocks :** À chaque clic sur un plat, le contrôleur demande l'ajout de l'élément dans le modèle (*Commande*). Cette action déclenche en cascade la méthode *plat.consommerIngredients()* pour déduire instantanément les matières premières de notre module de *stock*. Le modèle utilise ensuite l'objet d'écoute *CommandeEvenements.notifier*() pour ordonner au panier graphique de se rafraîchir.
3. **Validation et transmission :** Enfin, lorsque le serveur valide la commande, le contrôleur appelle *validerCommande*() et bascule l'état de préparation (*setPreparee*(*false*)). Un appel asynchrone vers *DaoJSON.sauvegarderArchitecture*() est immédiatement lancé pour figer les données sur le disque, tandis que le module cuisine reçoit la notification visuelle de la nouvelle commande à préparer.



C. Scénario technique : Cycle de persistance et gestion du stockage JSON

Le dernier diagramme de séquence explicite notre stratégie d'accès aux données locales (DAO) à travers les deux phases de vie principales de l'application :

1. **Chargement au démarrage :** Dès le lancement de la méthode *Main*, un appel est fait à *DaoJSON.chargerArchitecture*(*resto*). Notre classe *DaoJSON* fait office de passerelle et sollicite le *GestionnaireSauvegarde* afin de récupérer le restaurant actif. Ce gestionnaire va lire physiquement les fichiers textes architecture.json et *employes.json* stockés sur le disque de la machine pour reconstruire fidèlement en mémoire notre graphe d'objets Java (*tables*, *zones*, *plannings*).
2. **Sauvegarde après modification :** À chaque fois qu'un événement modifie l'état du restaurant (validation d'un planning, encaissement d'une addition, modification de stock), le contrôleur actif appelle *sauvegarderArchitecture*(). La couche *DAO* orchestre la sérialisation via *GSON* en traduisant les objets en chaînes *JSON* formatées, avant d'ordonner l'écriture sur le disque. L'isolation est assurée par le cloisonnement natif des fichiers par restaurant (*architecture.json*, *employes.json*, *planning.json*, *commandes.json*).



VI. CHOIX DE CONCEPTION, DIFFICULTÉS ET SOLUTIONS

A. Principaux choix de conception et de réalisation

L'implémentation de Manag'Eat a été guidée par la recherche d'une architecture modulaire et d'une expérience utilisateur fluide. Deux choix techniques majeurs caractérisent notre réalisation :

- **L'utilisation exclusive de Google GSON et du format JSON :** Plutôt que de mettre en place une base de données relationnelle lourde (type SQL) qui aurait complexifié le déploiement et l'installation de l'application chez le client final, nous avons opté pour une persistance par fichiers JSON locaux. La bibliothèque GSON nous permet de sérialiser et désérialiser instantanément des graphes d'objets complexes (incluant les plannings et l'historique des commandes) tout en conservant des fichiers de configuration facilement lisibles et éditables à la main en cas de besoin.

- **L'approche événementielle pour l'IHM (Java Swing) :** Pour l'interface graphique, le choix de Swing s'est imposé pour sa robustesse native sur les systèmes de bureau. Nous avons conçu une interface découpée en onglets métiers étanches, minimisant l'ouverture de fenêtres pop-up secondaires afin d'offrir une navigation fluide aux équipes en salle et en cuisine.

B. Problèmes techniques rencontrés et solutions apportées

Le développement collaboratif d'une application aussi dense que Manag'Eat a naturellement confronté l'équipe à plusieurs obstacles techniques majeurs. Ces défis, qu'ils soient liés à l'IHM, aux algorithmes ou à la persistance, ont nécessité une forte coordination pour être résolus.

- **Le défi de l'héritage et du polymorphisme avec GSON :** Lors des premiers essais de sauvegarde, le développeur en charge de la base de données s'est heurté à une limitation de la bibliothèque GSON. Lors de la relecture du fichier JSON, GSON ne parvenait pas à reconstruire automatiquement les sous-classes spécifiques (comme les différents types d'employés ou de plats) à partir des listes génériques du modèle. Pour résoudre ce problème, nous avons implémenté un adaptateur de type personnalisé (*RuntimeTypeAdapterFactory* de GSON). Cet outil injecte une métadonnée `"type"` directement dans les blocs JSON à l'écriture, ce qui force le transtypage correct et automatique des objets lors de la lecture.

- **La gestion du Drag & Drop et les conflits graphiques (Côté IHM et RH) :** Le binôme responsable du module RH a rencontré de grosses difficultés lors de la mise en place du calendrier interactif. Les déplacements de blocs horaires en Drag & Drop provoquaient des décalages visuels critiques, des superpositions de boutons et un gel de l'affichage Swing. La solution a consisté à isoler strictement la logique de collision dans le modèle (calcul des conflits d'horaires) et à forcer le rafraîchissement graphique via les méthodes `revalidate()` et `repaint()` appliquées directement par le contrôleur sur les conteneurs parents après chaque mouvement validé.

- **Le cloisonnement des données multi-restaurants :** Avec l'introduction de la carte de France en itération 2, un problème de fuite de données est apparu : le basculement d'un restaurant à un autre sur l'interface mélangeait parfois les commandes ou les plannings si les fichiers étaient mal synchronisés. L'équipe a dû restructurer la couche DAO en introduisant un identifiant unique d'établissement (`restaurantId`). Désormais, une vérification systématique de cet identifiant agit comme un filtre obligatoire sur toutes les requêtes de lecture et d'écriture, garantissant une étanchéité totale entre les restaurants de la chaîne.

- **La désynchronisation en temps réel de la Cuisine :** En itération 3, lors de la liaison des modules, l'envoi d'une commande depuis l'onglet salle n'apparaissait pas sur l'écran des cuisiniers à moins de redémarrer complètement l'application. Pour éviter d'avoir à recharger les fichiers JSON en boucle (ce qui aurait effondré les performances), nous avons mis en place le patron de conception *Observer/Observable*. Grâce à ce mécanisme d'écoute interne, dès qu'une commande est validée en salle, le modèle émet une notification qui pousse l'interface de la cuisine à se redessiner instantanément.

VII. ORGANISATION DE L'ÉQUIPE ET MÉTHODOLOGIE AGILE

A. Introduction et structuration du Backlog

Pour l'organisation et le suivi de notre travail, l'équipe a fait le choix d'appliquer la méthode agile SCRUM étudiée en cours. Le développement a ainsi été segmenté en plusieurs Sprints successifs, centralisés et pilotés à l'aide de l'outil de gestion de projet Jira.

Au sein de notre Backlog, le projet a été structuré selon une hiérarchie stricte afin de garantir une visibilité optimale sur l'arborescence logicielle. Les grandes briques fonctionnelles de Manag'Eat (tels que l'« Onglet Gestionnaire », l'« Onglet Dashboard » ou l'« Onglet Statistiques ») ont été définis comme des Epics. Nous avons ensuite découpé ces Epics en User Stories afin de cibler des fonctionnalités précises et concrètes du point de vue de l'utilisateur final (par exemple, la « Planification dynamique des équipes » ou l'« Automatisation de la Paie »).

Lors de nos séances de planification collectives, chaque User Story a fait l'objet d'une estimation de sa complexité en points d'effort, en suivant la suite classique de Fibonacci (1, 3, 5 ou 8). Enfin, dans le but de rendre le code plus digeste et d'attribuer des objectifs précis, chaque User Story a été scindée en au moins deux sous-tâches techniques très concrètes, accessibles en consultant le détail de la story parente. Cette hiérarchisation fine nous a permis de prioriser efficacement notre travail, en planifiant et en intégrant systématiquement les fonctionnalités les plus critiques dès les premiers Sprints.

B. Communication, exécution et transparence opérationnelle

Une fois le Sprint lancé, la répartition opérationnelle des tâches s'effectuait de manière concertée et naturelle. L'équipe s'appuyait quotidiennement sur l'utilisation du tableau de bord sur Jira.

Ce tableau offrait une transparence totale sur le flux de travail. D'un simple coup d'œil, chaque membre de l'équipe savait qui travaillait sur quel composant à un instant T (grâce aux colonnes À faire, En cours, Terminé). Pour fluidifier l'attribution et éviter les doublons ou les conflits dans la distribution des tâches, un système de réservation a été instauré : le développeur souhaitant prendre en charge un module s'assignait directement comme responsable de la User Story globale ou de la sous-tâche technique spécifique. Cette visibilité en temps réel s'est révélée cruciale, particulièrement lors des phases intenses d'écriture du code, et était continuellement synchronisée avec nos échanges directs sur notre groupe de communication WhatsApp.

C. Gestion de Git et protocole de prévention des conflits

Travaillant sur un dépôt de code Git commun, le risque principal identifié par l'équipe était l'apparition de conflits majeurs ou l'écrasement accidentel du travail d'un camarade lors des fusions (merges). Pour sécuriser nos branches de développement, nous avons mis en place un protocole de communication rigoureux et systématique :

1- Alerte de début de tâche : Avant de modifier le moindre fichier en local, le développeur concerné devait obligatoirement déplacer sa User Story dans la colonne « En cours » sur Jira. Simultanément, il envoyait un message d'alerte sur le groupe WhatsApp pour prévenir le reste de l'équipe qu'il modifiait une portion spécifique du code.

2- Alerte de fin et validation : Une fois le développement terminé, testé et déposé (pushé) sur Git, un dernier message de confirmation était envoyé sur WhatsApp. Ce message résumait explicitement les ajouts effectués et donnait le « feu vert » aux autres membres pour mettre à jour (pull) leur version locale en toute sécurité.

Ce double verrouillage (Jira + WhatsApp) a permis de maintenir une base de code stable tout au long des itérations, sans aucune perte de données.

D. Documentation et livrables communs

En marge du développement technique, la rédaction des livrables (rapports généraux, spécifications et manuel d'utilisation) a également fait l'objet d'une approche collaborative. Nous avons centralisé nos documents sur un espace partagé Google Docs, ce qui a permis à l'ensemble du groupe de travailler simultanément sur les mêmes fichiers.

Afin de garantir une parfaite cohérence de style et de contenu dans la version finale du projet, nous avons organisé des séances de rédaction communes en temps réel à des heures définies. Ce travail synchrone a grandement facilité la relecture croisée et la validation collective de la documentation, assurant un rendu final homogène et complet.

VIII. Conclusion

Ce projet de conception et de développement de la plateforme de gestion de restauration Manag'Eat nous a permis de mettre en œuvre une application de bureau complète, modulaire et intuitive, répondant aux exigences d'exploitation réelles des établissements modernes. Nous avons réussi à faire évoluer les briques fonctionnelles de base, centrées sur le terrain, vers un système complet et centralisé capable de piloter à la fois la salle, les cuisines, la gestion des ressources humaines et le suivi multi-restaurants.

L'implémentation du patron d'architecture MVC, couplée à une couche de persistance DAO basée sur la sérialisation JSON (Google GSON), a constitué le pilier technique du projet. Ce choix a offert une alternative légère et performante aux bases de données lourdes, tout en résolvant des défis majeurs tels que la gestion du polymorphisme des données et l'isolation stricte entre les différents établissements de la chaîne. Ce travail a confirmé l'importance d'un découpage rigoureux en paquetages étanches et d'une conception événementielle maîtrisée pour garantir la robustesse du logiciel face à des scénarios d'utilisation intensifs et synchrones (salle et cuisine).

Au-delà de l'aspect purement algorithmique et architectural, cette expérience nous a confrontés aux réalités de la gestion de projet et de l'ingénierie collaborative. L'application concrète de la méthode agile SCRUM via Jira, associée à une discipline de communication rigoureuse sur nos canaux d'équipe pour la gestion des branches Git, a été la clé pour éviter la dette technique et les conflits de code. En conclusion, ce projet a développé notre aptitude à concevoir des architectures logicielles fiables, évolutives et prêtes pour de futures migrations technologiques, tout en nous sensibilisant aux enjeux de synchronisation et de cohésion qu'implique un véritable travail d'équipe.